

RDS TASK

Task 1

Create a MariaDB Database on an EC2 Instance

Objective

Deploy a MariaDB server on an EC2 instance to store application data and prepare for later migration to RDS.

Prerequisites

Before starting, ensure the following:

- **VPC** (default or custom)
- **Public Subnet** (for EC2)
- **Internet Gateway** (attached to VPC)
- **Route Table** with route `0.0.0.0/0 → IGW`
- **Security Group**
 - Allow **22 (SSH)** from your IP
 - Allow **3306 (MariaDB)** from your IP
- **Key Pair** (PEM file)
- **EC2 Instance** using **Amazon Linux 2 AMI**
- **IAM Role** (optional – only needed if backing up to S3)
- **Package Requirement:** MariaDB server

Step-by-Step Implementation (With Evidence)

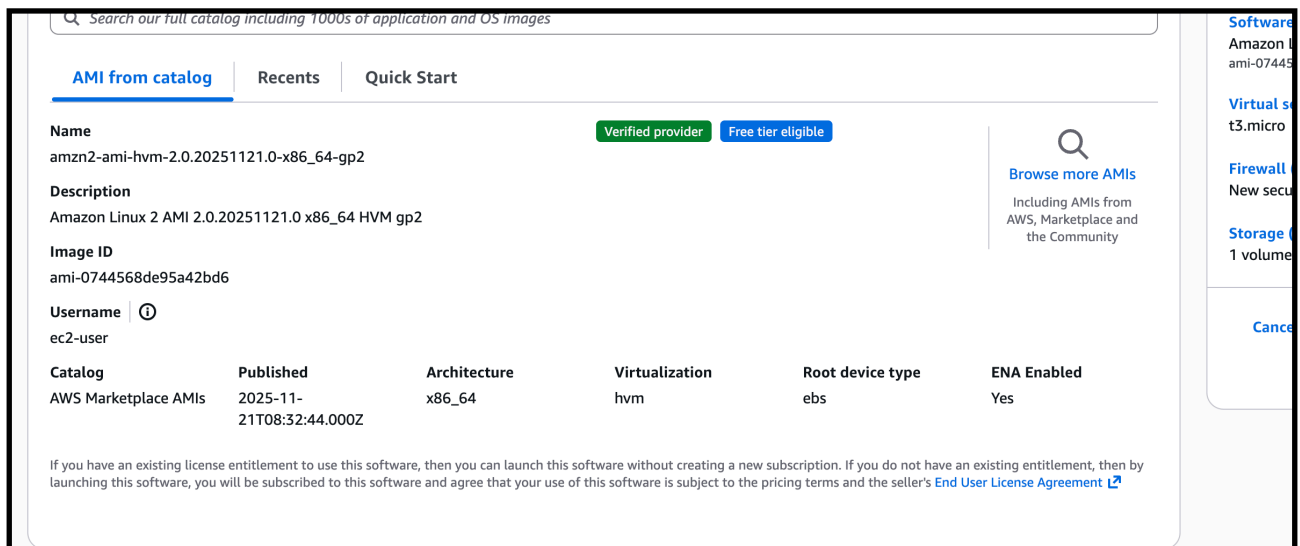
Step 1: Launch an EC2 Instance

Console Path:

EC2 → Instances → Launch Instance

Configuration:

- AMI: **Amazon Linux 2**
- Instance Type: **t2.micro / t3.micro**
- Key Pair: Your key
- Security Group:
 - SSH (22) → My IP
 - MariaDB (3306) → My IP
- Storage: 8–10 GB (default ok)



—-> select the vpc and remember the subnet which you have taken

▼ Network settings

Info

VPC - required

Info

vpc-01126d967b1bb73af

172.31.0.0/16

(default)

↻

Subnet

Info

subnet-0a094896b5cb2db9e

public-subnet

VPC: vpc-01126d967b1bb73af

Owner: 207662791773

Availability Zone: eu-north-1a (eun1-az1)

Zone type: Availability Zone

IP addresses available: 4091

CIDR: 172.31.16.0/20

↻

Create new subnet

Auto-assign public IP

Info

Enable

▼

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

Security group name - required

Amazon Linux 2 AMI (HVM), SSD Volume Type (64-bit x86) Operating System-2.0.20251121.0-Autogeni

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-:/()#,@[]+=&;{}!\$*

Description - required

Info

Amazon Linux 2 AMI (HVM), SSD Volume Type (64-bit x86) Operating System-2.0.20251121.0-Autogeni

Inbound Security Group Rules

▼ Security group rule 1 (TCP, 22, 0.0.0.0/0)

Remove

EC2 > Instances

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Instances (1/1)

Info

Last updated 14 minutes ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

☒	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 .
☒	mariadb-ec2	i-00dbf80720d77bab6	Running	t3.micro	3/3 checks passed	View alarms +	eu-north-1a	ec2-13-49-46-180.eu-n...	13.49.46.180

Step 2: Install MariaDB Server

```

sudo yum update -y
sudo yum install mariadb-server -y
sudo systemctl enable mariadb
sudo systemctl start mariadb

```


—> sudo systemctl status mariadb : shows the status of the mariadb server

```
[root@ip-172-31-21-190 ~]# sudo systemctl enable mariadb
Created symlink from /etc/systemd/system/multi-user.target.wants/mariadb.service to /usr/lib/systemd/system/mariadb.service.
[root@ip-172-31-21-190 ~]# sudo systemctl start mariadb
[root@ip-172-31-21-190 ~]# sudo systemctl status mariadb
● mariadb.service - MariaDB database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; vendor preset: disabled)
   Active: active (running) since Fri 2025-12-12 09:50:21 UTC; 1min 20s ago
     Process: 5730 ExecStartPost=/usr/libexec/mariadb-wait-ready $MAINPID (code=exited, status=0/SUCCESS)
     Process: 5638 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir %n (code=exited, status=0/SUCCESS)
    Main PID: 5729 (mysqld_safe)
      CGroup: /system.slice/mariadb.service
              └─5729 /bin/sh /usr/bin/mysqld_safe --basedir=/usr
                  └─5895 /usr/libexec/mysqld --basedir=/usr --datadir=/var/lib/mysql --plugin-dir=/usr/lib64/mysql/plugin --log-error=/var/log/mariadb/mariadb.log --pid-
```

```
Dec 12 09:50:17 ip-172-31-21-190.eu-north-1.compute.internal systemd[1]: Starting MariaDB database server...
Dec 12 09:50:17 ip-172-31-21-190.eu-north-1.compute.internal mariadb-prepare-db-dir[5638]: Initializing MariaDB database
Dec 12 09:50:18 ip-172-31-21-190.eu-north-1.compute.internal mariadb-prepare-db-dir[5638]: 251212 9:50:18 [Note] /usr/libexec/mysqld (mysqld 5.5.68-MariaDB) star
Dec 12 09:50:18 ip-172-31-21-190.eu-north-1.compute.internal mariadb-prepare-db-dir[5638]: 251212 9:50:18 [Note] /usr/libexec/mysqld (mysqld 5.5.68-MariaDB) star
Dec 12 09:50:19 ip-172-31-21-190.eu-north-1.compute.internal mariadb-prepare-db-dir[5638]: PLEASE REMEMBER TO SET A PASSWORD FOR THE MariaDB root USER !
Dec 12 09:50:19 ip-172-31-21-190.eu-north-1.compute.internal mariadb-prepare-db-dir[5638]: To do so, start the server, then issue the following commands:
Dec 12 09:50:19 ip-172-31-21-190.eu-north-1.compute.internal mysqld_safe[5729]: 251212 09:50:19 mysqld_safe Logging to '/var/log/mariadb/mariadb.log'.
Dec 12 09:50:19 ip-172-31-21-190.eu-north-1.compute.internal mysqld_safe[5729]: 251212 09:50:19 mysqld_safe Starting mysqld daemon with databases from /var/lib/my
Dec 12 09:50:21 ip-172-31-21-190.eu-north-1.compute.internal systemd[1]: Started MariaDB database server.
[root@ip-172-31-21-190 ~]#
```

Step 3: Secure MariaDB

—> Sudo mysql_secure_installation

Choose:

- Set root password → YES
- Remove anonymous users → YES
- Disallow remote login → YES
- Remove test DB → YES

```
[root@ip-172-31-21-190 ~]# sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
you haven't set the root password yet, the password will be blank,
so you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password ensures that nobody can log into the MariaDB
root user without the proper authorisation.

Set root password? [Y/n] y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n] y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
[root@ip-172-31-21-190 ~]#
```

Validation : DB login : `sudo mysql -u root -p`

```
Thanks for using MariaDB!
[root@ip-172-31-21-190 ~]# sudo mysql -u root -p
[Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 5.5.68-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Conclusion

MariaDB was successfully installed and configured on EC2.

TASK 2 — Insert Dummy Data into MariaDB

—> Task Title

Insert Dummy Data into MariaDB Database on EC2

Objective

Populate the created MariaDB database with sample records for testing and migrations.

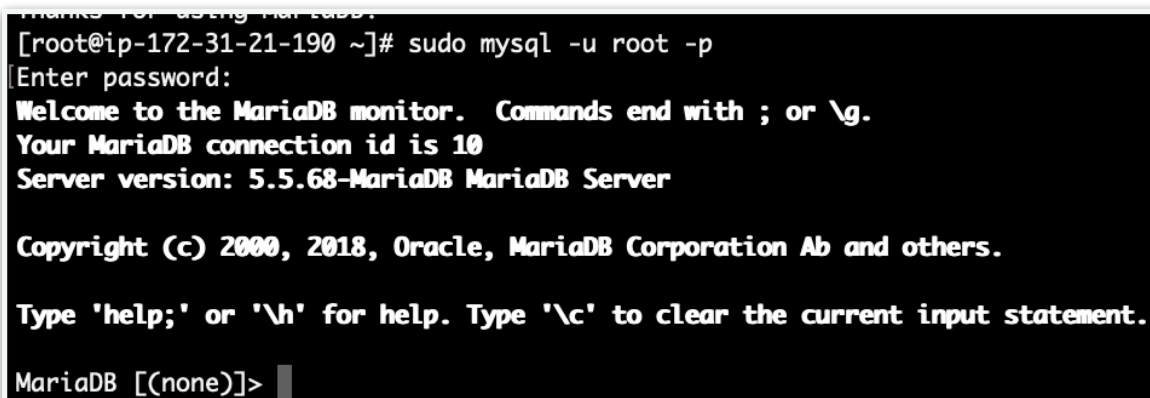
Prerequisites

- Task 1 completed
- MariaDB running on EC2
- Database `companydb` exists
- MariaDB root password (if set)

Step-by-Step Implementation

Step 1: Login to MariaDB

```
sudo mysql -u root -p
```

A terminal window with a black background and white text. The text shows the execution of 'sudo mysql -u root -p', followed by a password prompt. After login, it displays the MariaDB monitor welcome message, connection ID 10, and server version 5.5.68. It also shows copyright information and help instructions. The prompt 'MariaDB [(none)]>' is visible at the bottom.

```
Thanks for using MariaDB!  
[root@ip-172-31-21-190 ~]# sudo mysql -u root -p  
[Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 10  
Server version: 5.5.68-MariaDB MariaDB Server  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
MariaDB [(none)]> █
```

Step 2: Create Database ,”CREATE DATABASE companydb”

```
CREATE DATABASE companydb;
```

Step 3: Select Database

```
USE companydb;
```

Step 4: Create Table

```
CREATE TABLE employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50),  
    role VARCHAR(50),  
    salary INT  
);
```

Step 5: Insert Dummy Data

```
INSERT INTO employees (name, role, salary) VALUES
('Waseem', 'DevOps Engineer', 50000),
('Akram', 'Cloud Engineer', 60000),
('John', 'Developer', 45000),
('Sarah', 'Tester', 42000);
```

```
MariaDB [(none)]> CREATE DATABASE companydb;
Query OK, 1 row affected (0.00 sec)

MariaDB [(none)]> USE companydb;
Database changed
MariaDB [companydb]> CREATE TABLE employees (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> name VARCHAR(50),
  -> role VARCHAR(50),
  -> salary INT
  -> );
Query OK, 0 rows affected (0.01 sec)

MariaDB [companydb]> INSERT INTO employees (name, role, salary) VALUES
  -> ('Waseem', 'DevOps Engineer', 50000),
  -> ('Akram', 'Cloud Engineer', 60000),
  -> ('John', 'Developer', 45000),
  -> ('Sarah', 'Tester', 42000);
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

MariaDB [companydb]> SELECT * FROM employees;
+-----+-----+-----+-----+
| id | name | role | salary |
+-----+-----+-----+-----+
| 1 | Waseem | DevOps Engineer | 50000 |
| 2 | Akram | Cloud Engineer | 60000 |
| 3 | John | Developer | 45000 |
| 4 | Sarah | Tester | 42000 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

MariaDB [companydb]> █
```

Step 6: Verify Data

```
SELECT * FROM employees;
```

Conclusion

Dummy data successfully inserted into the MariaDB table for further backup and migration.

TASK 3 — Take Backup of Dummy Data on EC2

① Task Title

Backup MariaDB Dummy Data on EC2 Using mysqldump

② Objective

Create a portable SQL backup file to use for RDS migration or disaster recovery.

③ Prerequisites

- Task 1 & 2 completed
- MariaDB service running
- EC2 Linux shell access
- Sufficient disk space

—->Step-by-Step Implementation

Step 1: Exit MariaDB Shell

If currently inside:

Step 2: Take Backup Using mysqldump

If root has password:

```
sudo mysqldump -u root -p companydb > /home/ec2-user/  
companydb_backup.sql
```

—>enter the password for root

```
exit at line 1  
MariaDB [companydb]> exit;  
Bye  
[root@ip-172-31-21-190 ~]# sudo mysqldump -u root -p companydb > /home/ec2-user/companydb_backup.sql  
[Enter password]:  
[root@ip-172-31-21-190 ~]# ls -lh /home/ec2-user/companydb_backup.sql  
-rw-r--r-- 1 root root 2.1K Dec 12 10:01 /home/ec2-user/companydb_backup.sql  
[root@ip-172-31-21-190 ~]#
```

Step 3: Verify Backup File

```
ls -lh /home/ec2-user/companydb_backup.sql
```

Validation Steps

✓ **Validation 1: Backup file exists**

```
[root@ip-172-31-21-190 ~]# ls -lh /home/ec2-user/companydb_backup.sql  
-rw-r--r-- 1 root root 2.1K Dec 12 10:01 /home/ec2-user/companydb_backup.sql  
[root@ip-172-31-21-190 ~]#
```

TASK 4 — Launch MariaDB RDS Instance

① Task Title

Launch a MariaDB RDS Instance in AWS

② Objective

Set up a fully managed MariaDB database using Amazon RDS to improve reliability, automated backups, and easier migrations from EC2.

③ Prerequisites

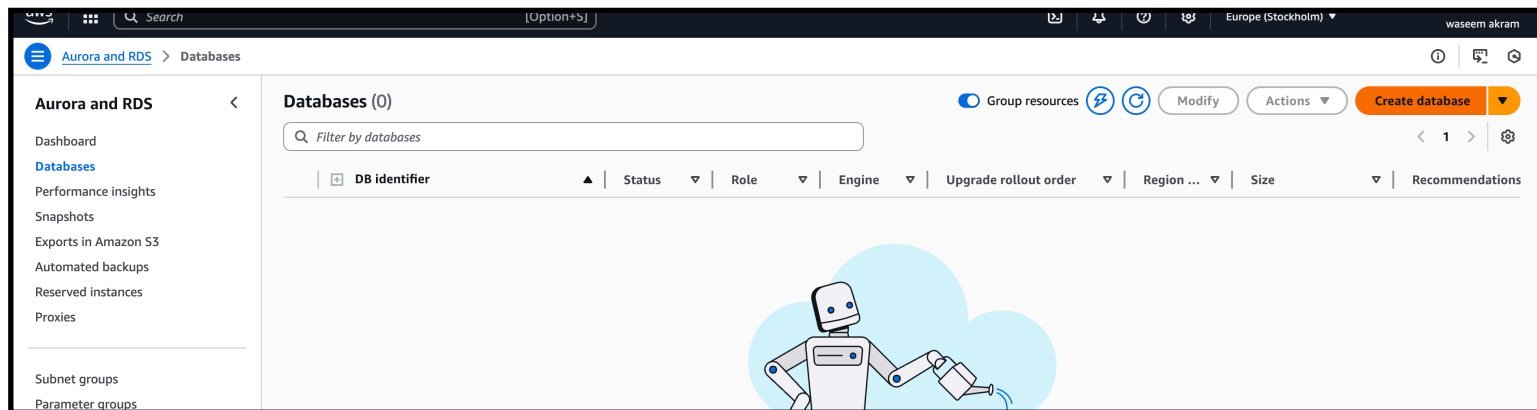
- VPC
- At least **2 subnets** in different AZs
- **DB Subnet Group** (or create one during steps)
- Security Group allowing inbound **3306** from EC2 instance
- IAM role (optional)
- EC2 instance with backup SQL file ready
- MariaDB client installed on EC2
- Parameter group (optional)
- Storage size requirement understood

—>Step-by-Step Implementation (With Evidence)

—> Step 1: Open RDS Console

Console Path:

AWS Console → RDS → Databases → **Create database**

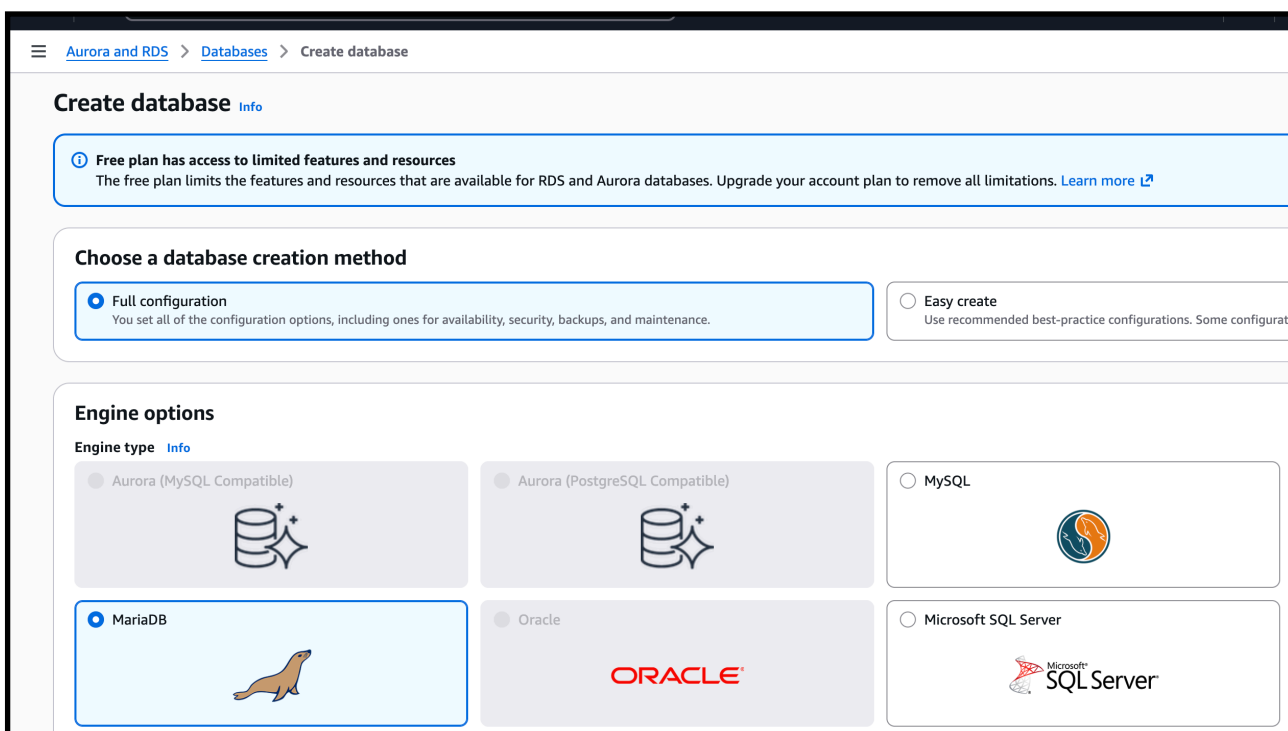


Step 2: Select Database Creation Method

Choose ; full configuration

—> **Choose Engine Type**

- **MariaDB**



Step 3: Version Selection

Choose a stable version:

Engine version

MariaDB 11.4.8

Templates

Choose a sample template to meet your use case.

☐ Production

Use defaults for high availability and fast, consistent performance.

☐ Dev/Test

This instance is intended for development use outside of a production environment.

☒ Free tier

Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.

Settings

DB instance identifier

Info

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

database-1

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Step 4: DB Instance Settings

Fill the fields:

Field	Value
DB Instance identifier	mariadb-rds-instance
Master username	admin
Master password	*****
Confirm password	*****

▼ Credentials Settings

Master username

Info

Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - *most secure*

RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed

Create your own password or have RDS create a password that you manage.

☐ Auto generate password

Amazon RDS can generate a password for you, or you can specify your own password.

Master password

Info

Password strength

Strong

Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / ' " @

Confirm master password

Info

Step 5: Configure Instance Type

Choose:

db.t3.micro (Free tier eligible)

Instance configuration

The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class

[Info](#)

▼ Hide filters

☐ Show instance classes that support Amazon RDS Optimized Writes

[Info](#)

Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

☐ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

db.t3.micro

2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2,085 Mbps Network: Up to 5 Gbps

Storage

Storage type

[Info](#)

Provisioned IOPS SSD (io2) storage volumes are now available.

General Purpose SSD (gp2)

Baseline performance determined by volume size

Allocated storage

[Info](#)

20

GiB

Allocated storage value must be 20 GiB to 6,144 GiB

► Additional storage configuration

Compute resource

Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ Don't connect to an EC2 compute resource

Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ Connect to an EC2 compute resource

Set up a connection to an EC2 compute resource for this database.

Network type

[Info](#)

To use dual-stack mode, make sure that you associate an IPv6 CIDR block with a subnet in the VPC you specify.

☒ IPv4

Your resources can communicate only over the IPv4 addressing protocol.

☐ Dual-stack mode

Your resources can communicate over IPv4, IPv6, or both.

Virtual private cloud (VPC)

[Info](#)

Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Default VPC (vpc-01126d967b1bb73af)

3 Subnets, 3 Availability Zones

Only VPCs with a corresponding DB subnet group are listed.

ⓘ After a database is created, you can't change its VPC.

DB subnet group

[Info](#)

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

default

Public access

[Info](#)

☐ Yes

RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect

Step 9: Connectivity Settings

a) Choose VPC

- Default VPC OR your custom VPC

b) DB Subnet Group

- Select **default** or **your DB subnet group**

c) Public Access

Choose:

- **No** (recommended)
or
- **Yes** (if you want to connect from your laptop)

DB subnet group [Info](#)

Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

default

Public access [Info](#)

☐ **Yes**
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☒ **No**
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)

Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☒ **Choose existing**
Choose existing VPC security groups

☐ **Create new**
Create new VPC security group

Existing VPC security groups

Choose one or more options

default X

Availability Zone [Info](#)

eu-north-1a

RDS Proxy

RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ **Create an RDS Proxy** [Info](#)
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Choose one or more options

default X

Availability Zone [Info](#)

eu-north-1a

RDS Proxy

RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ **Create an RDS Proxy** [Info](#)
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

Certificate authority - optional [Info](#)

Using a server certificate provides an extra layer of security by validating that the connection is being made to an Amazon database. It does so by checking the server certificate that is automatically installed on all databases that you provision.

rds-ca-rsa2048-g1 (default)
Expiry: May 25, 2061

If you don't select a certificate authority, RDS chooses one for you.

► **Additional configuration**

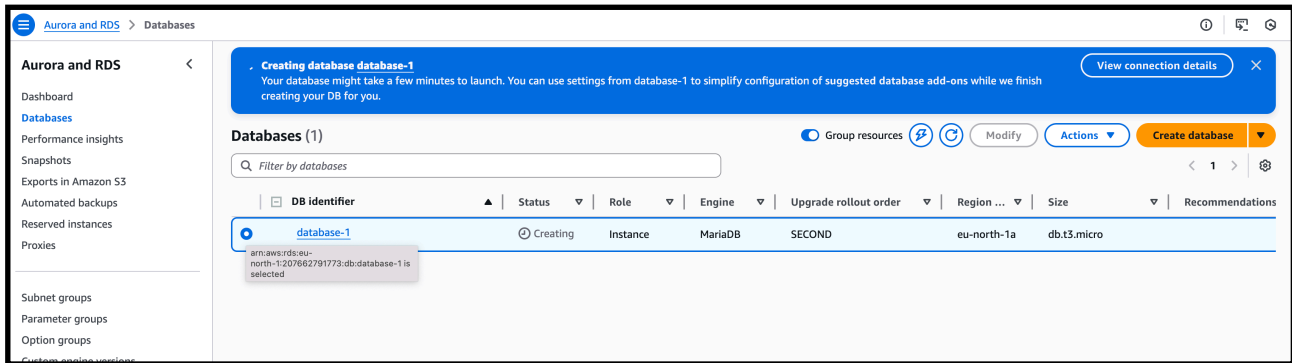
Tags - optional

A tag consists of a case-sensitive key-value pair.

No tags associated with the resource.

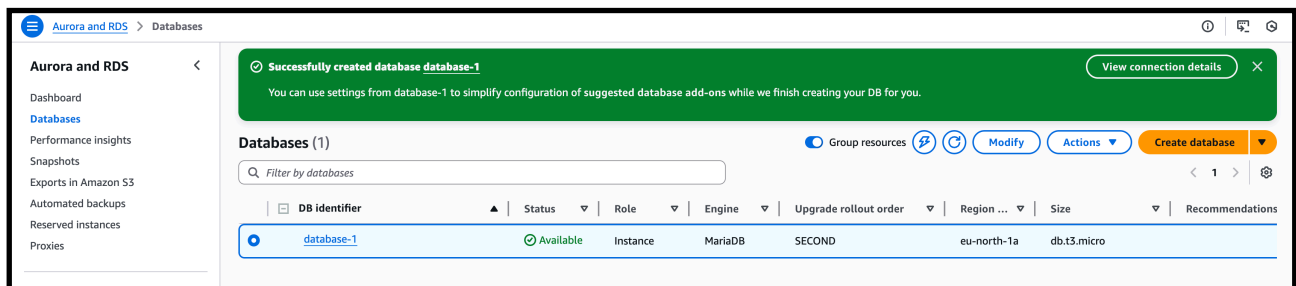
Step 11: Create Database

Click: create



—> Wait Until Status = "Available"

The DB takes 5–10 minutes.



—>Validation

Connectivity Test from EC2

```
mysql -h database-1.ct8mquyak19o.eu-north-1.rds.amazonaws.com -u admin -p
```

```
[root@ip-172-31-21-190 ~]# mysql -h database-1.ct8mquyak19o.eu-north-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 81
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Validation 2: Check DB Created

Inside MySQL:

```
SHOW DATABASES;
```

```
Copyright (C) 2000, 2018, Oracle, MariaDB  
Type 'help;' or '\h' for help. Type '\c'  
MariaDB [(none)]> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| companydb |  
| information_schema |  
| innodb |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
6 rows in set (0.00 sec)  
MariaDB [(none)]> █
```

Conclusion

A fully functional MariaDB RDS instance was launched, configured, and validated successfully. It is now ready for database migration from the EC2-hosted MariaDB instance.

TASK 5 — Migrate Database from EC2 to RDS

① Task Title

Migrate MariaDB Database from EC2 Instance to MariaDB RDS Instance

② Objective

Transfer the MariaDB database hosted on an EC2 instance to a fully managed Amazon RDS MariaDB instance using mysqldump and restore.

③ Prerequisites

- EC2 instance with MariaDB installed
- EC2 contains database **companydb**
- Backup file created: **companydb_backup.sql**
- RDS MariaDB instance created and **Available**
- RDS endpoint copied
- Security Group updated to allow EC2 → RDS on port **3306**
- MariaDB client installed on EC2
- Correct credentials for RDS (admin user)

④ Step-by-Step Implementation (With Evidence)

Step 1: Verify Backup File on EC2

Command:

```
ls -lh /home/ec2-user/companydb_backup.sql
```

```
[root@ip-172-31-21-190 ~]# ls -lh /home/ec2-user/companydb_backup.sql
-rw-r--r-- 1 root root 2.1K Dec 12 10:01 /home/ec2-user/companydb_backup.sql
```

Step 2: Test Connection from EC2 to RDS

Command:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

```
mysql -h database-1.ct8mquyak19o.eu-north-1.rds.amazonaws.com -u admin -p
```

—> copy RDS-ENDPOINT from the RDS from ec2

The screenshot displays the AWS Management Console interface for an Amazon RDS database instance. At the top, a green banner indicates 'Successfully created database database-1'. Below this, the instance details are shown for 'database-1'.

Summary:

- DB identifier: database-1
- Status: Available
- Role: Instance
- Engine: MariaDB
- CPU: 4.14%
- Class: db.t3.micro
- Current activity: 0 Connections
- Region & AZ: eu-north-1a

Connectivity & security:

- Endpoint: database-1.ct8mquyak19o.eu-north-1.rds.amazonaws.com
- Port: 3306
- Availability Zone: eu-north-1a
- VPC: vpc-01126d967b1bb73af
- Subnet group: default-vpc-01126d967b1bb73af
- Subnets: subnet-006e395d3b72e2ed7, subnet-05f27e3702839a75e, subnet-0a094896b5cb2db9e
- VPC security groups: default (sg-0853619dc44ef762e) - Active
- Publicly accessible: No
- Certificate authority: rds-ca-rsa2048-g1
- Certificate authority date: May 25, 2061, 03:29 (UTC+05:30)
- DB instance certificate expiration date: December 12, 2026, 16:12 (UTC+05:30)

Step 3: Create Empty Database on RDS

Inside RDS MariaDB shell:

```
CREATE DATABASE companydb;
```

```
[root@ip-172-31-21-190 ~]# mysql -h database-1.ct8mqyak19o.eu-north-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 39
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE companydb;
Query OK, 1 row affected (0.01 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| companydb |
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.00 sec)

MariaDB [(none)]> █
```

Step 4: Migrate Data Using mysqldump Restore

Run this on **EC2 shell**, NOT inside MariaDB:

```
mysql -h database-1.ct8mqyak19o.eu-north-1.rds.amazonaws.com -u admin -p companydb < /
home/ec2-user/companydb_backup.sql
```

```
[root@ip-172-31-21-190 ~]# mysql -h database-1.ct8mqyak19o.eu-north-1.rds.amazonaws.com -u admin -p companydb < /home/ec2-user/companydb_backup.sql
Enter password:
[root@ip-172-31-21-190 ~]#
```

This imports all tables and data into RDS.

Step 5: Validate Data on RDS

Login to RDS:

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

```
[root@ip-172-31-21-190 ~]# mysql -h database-1.ct8mqyak19o.eu-north-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 101
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE companydb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [companydb]> SELECT * FROM employees;
+----+-----+-----+-----+
| id | name  | role          | salary |
+----+-----+-----+-----+
| 1  | Waseem | DevOps Engineer | 50000  |
| 2  | Akram  | Cloud Engineer  | 60000  |
| 3  | John   | Developer       | 45000  |
| 4  | Sarah  | Tester         | 42000  |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

MariaDB [companydb]>
```

Then:

```
--> USE companydb;
--> SELECT * FROM employees;
```

Step 6: Compare Data Between EC2 and RDS (Optional but recommended)

On EC2:

```
mysql -u root -p -e "SELECT COUNT(*) FROM companydb.employees;"
```

```
[root@ip-172-31-21-190 ~]# mysql -u root -p -e "SELECT COUNT(*) FROM companydb.employees;"
Enter password:
+-----+
| COUNT(*) |
+-----+
|          4 |
+-----+
```

Validation Steps (With Evidence)

✓ Validation 1: RDS Connection Test

```
mysql -h <RDS-ENDPOINT> -u admin -p successful
```

```
[root@ip-172-31-21-190 ~]# mysql -h database-1.ct8mqyak19o.eu-north-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 101
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Validation 2: Database Exists on RDS,Table Structure Matches

```
MariaDB [(none)]> SHOW DATABASES;
```

```
+-----+
| Database |
+-----+
| companydb |
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.00 sec)
```

```
MariaDB [(none)]> █
```

SHOW DATABASES;

DESCRIBE employees;

```
[MariaDB [companydb]> SHOW DATABASES;
```

```
+-----+
| Database |
+-----+
| companydb |
| information_schema |
| innodb |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set (0.00 sec)
```

```
[MariaDB [companydb]> SELECT * FROM employees;
```

```
+-----+-----+-----+-----+
| id | name | role | salary |
+-----+-----+-----+-----+
| 1 | Waseem | DevOps Engineer | 50000 |
| 2 | Akram | Cloud Engineer | 60000 |
| 3 | John | Developer | 45000 |
| 4 | Sarah | Tester | 42000 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
[MariaDB [companydb]> DESCRIBE employees;
```

```
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| name | varchar(50) | YES | | NULL | |
| role | varchar(50) | YES | | NULL | |
| salary | int(11) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
MariaDB [companydb]> █
```

Conclusion

The MariaDB database was successfully migrated from EC2 to RDS using mysqldump and restore. All tables and data were verified, and connectivity was confirmed between EC2 and RDS.

TASK 6 — Install MySQL Database on EC2

① Task Title

Install MySQL Server on an EC2 Instance

② Objective

Deploy a MySQL database on an EC2 instance for application usage, testing, or migration to RDS MySQL.

③ Prerequisites

- VPC
- Public Subnet
- Internet Gateway + Route Table
- Key Pair (PEM)
- Security Group allowing:
 - SSH (22) → My IP
 - MySQL (3306) → My IP or EC2 SG
- EC2 instance (Amazon Linux 2023 or Amazon Linux 2)
- System packages updated

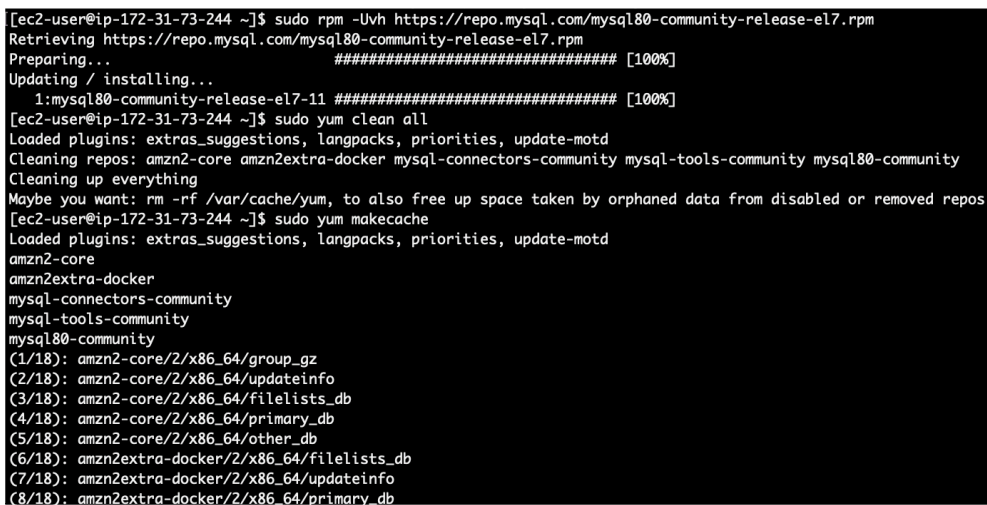
- IAM role (optional)
- Enough storage (8GB+)

④ Step-by-Step Implementation (With Evidence)

STEP 1 — Add the MySQL 8.0 Repository

```
→ sudo rpm -Uvh https://repo.mysql.com/mysql80-community-release-el7.rpm
```

This installs the official MySQL YUM repo for EL7 (compatible with Amazon Linux 2).



```
[ec2-user@ip-172-31-73-244 ~]$ sudo rpm -Uvh https://repo.mysql.com/mysql80-community-release-el7.rpm
Retrieving https://repo.mysql.com/mysql80-community-release-el7.rpm
Preparing... ##### [100%]
Updating / installing...
 1:mysql80-community-release-el7-11 ##### [100%]
[ec2-user@ip-172-31-73-244 ~]$ sudo yum clean all
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
Cleaning repos: amzn2-core amzn2extra-docker mysql-connectors-community mysql-tools-community mysql80-community
Cleaning up everything
Maybe you want: rm -rf /var/cache/yum, to also free up space taken by orphaned data from disabled or removed repos
[ec2-user@ip-172-31-73-244 ~]$ sudo yum makecache
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core
amzn2extra-docker
mysql-connectors-community
mysql-tools-community
mysql80-community
(1/18): amzn2-core/2/x86_64/group_gz
(2/18): amzn2-core/2/x86_64/updateinfo
(3/18): amzn2-core/2/x86_64/filelists_db
(4/18): amzn2-core/2/x86_64/primary_db
(5/18): amzn2-core/2/x86_64/other_db
(6/18): amzn2extra-docker/2/x86_64/filelists_db
(7/18): amzn2extra-docker/2/x86_64/updateinfo
(8/18): amzn2extra-docker/2/x86_64/primary_db
```

STEP 2 — Clean yum metadata

From screenshot:

```
sudo yum clean all
```

STEP 3 — Rebuild yum cache

Your friend ran:

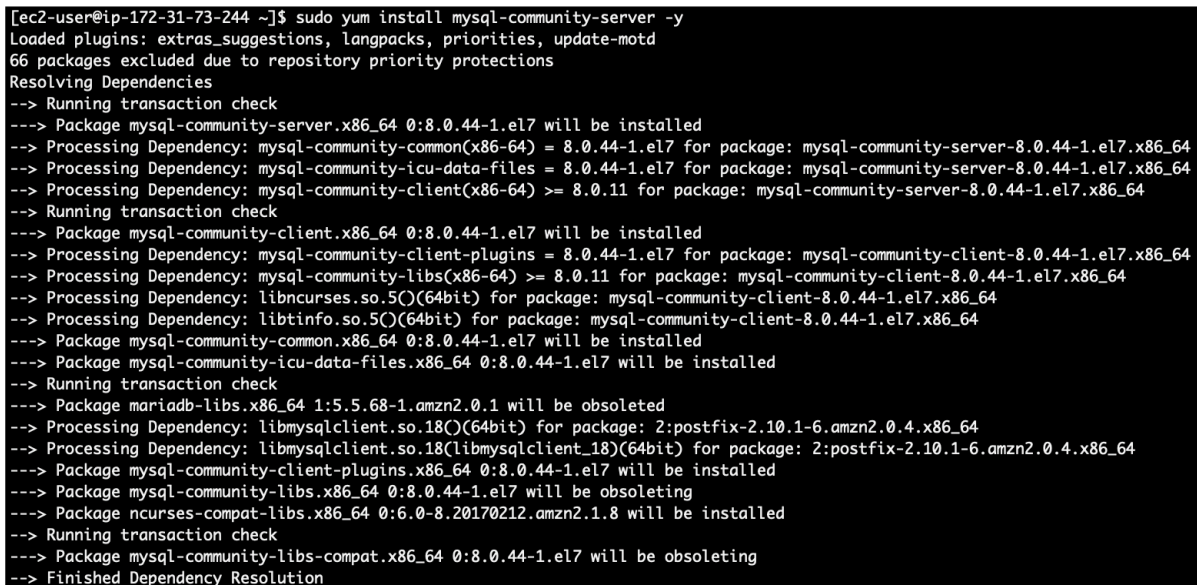
```
sudo yum makecache
```

—> This loads all repositories including mysql80-community.

STEP 4 — Install MySQL 8.0 Server

Based on the screenshot:

```
sudo yum install mysql-community-server -y
```



```
[ec2-user@ip-172-31-73-244 ~]$ sudo yum install mysql-community-server -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
66 packages excluded due to repository priority protections
Resolving Dependencies
--> Running transaction check
--> Package mysql-community-server.x86_64 0:8.0.44-1.el7 will be installed
--> Processing Dependency: mysql-community-common(x86-64) = 8.0.44-1.el7 for package: mysql-community-server-8.0.44-1.el7.x86_64
--> Processing Dependency: mysql-community-icu-data-files = 8.0.44-1.el7 for package: mysql-community-server-8.0.44-1.el7.x86_64
--> Processing Dependency: mysql-community-client(x86-64) >= 8.0.11 for package: mysql-community-server-8.0.44-1.el7.x86_64
--> Running transaction check
--> Package mysql-community-client.x86_64 0:8.0.44-1.el7 will be installed
--> Processing Dependency: mysql-community-client-plugins = 8.0.44-1.el7 for package: mysql-community-client-8.0.44-1.el7.x86_64
--> Processing Dependency: mysql-community-libs(x86-64) >= 8.0.11 for package: mysql-community-client-8.0.44-1.el7.x86_64
--> Processing Dependency: libncurses.so.5()(64bit) for package: mysql-community-client-8.0.44-1.el7.x86_64
--> Processing Dependency: libtinfo.so.5()(64bit) for package: mysql-community-client-8.0.44-1.el7.x86_64
--> Package mysql-community-common.x86_64 0:8.0.44-1.el7 will be installed
--> Package mysql-community-icu-data-files.x86_64 0:8.0.44-1.el7 will be installed
--> Running transaction check
--> Package mariadb-libs.x86_64 1:5.5.68-1.amzn2.0.1 will be obsoleted
--> Processing Dependency: libmysqlclient.so.18()(64bit) for package: 2:postfix-2.10.1-6.amzn2.0.4.x86_64
--> Processing Dependency: libmysqlclient.so.18(libmysqlclient_18)(64bit) for package: 2:postfix-2.10.1-6.amzn2.0.4.x86_64
--> Package mysql-community-client-plugins.x86_64 0:8.0.44-1.el7 will be installed
--> Package mysql-community-libs.x86_64 0:8.0.44-1.el7 will be obsoleting
--> Package ncurses-compat-libs.x86_64 0:6.0-8.20170212.amzn2.1.8 will be installed
--> Running transaction check
--> Package mysql-community-libs-compat.x86_64 0:8.0.44-1.el7 will be obsoleting
--> Finished Dependency Resolution
```

This installed:

- mysql-community-server
- mysql-community-client
- mysql-community-libs
- ncurses compat libs
- MySQL common files

It *replaced* MariaDB installations automatically.

STEP 5 — Start MySQL Service

Your friend ran:

```
sudo systemctl start mysqld
sudo systemctl enable mysqld
```

STEP 7 — Check MySQL Service Status

Your screenshot shows:

```
sudo systemctl status mysqld
```

And the output was:

Active: active (running)
Meaning MySQL is successfully running.

```
Installed:
  mysql-community-libs.x86_64 0:8.0.44-1.el7                                mysql-

Dependency Installed:
  mysql-community-client.x86_64 0:8.0.44-1.el7      mysql-community-client-plugins.x86_64 0:8.0.44

Replaced:
  mariadb-libs.x86_64 1:5.5.68-1.amzn2.0.1

Complete!
[ec2-user@ip-172-31-73-244 ~]$ sudo systemctl start mysqld

[ec2-user@ip-172-31-73-244 ~]$
[ec2-user@ip-172-31-73-244 ~]$ sudo systemctl enable mysqld
[ec2-user@ip-172-31-73-244 ~]$ sudo systemctl status mysqld
● mysqld.service - MySQL Server
   Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; vendor preset: disabled)
   Active: active (running) since Fri 2025-12-12 12:43:05 UTC; 39s ago
     Docs: man:mysqld(8)
           http://dev.mysql.com/doc/refman/en/using-systemd.html
   Main PID: 5884 (mysqld)
     Status: "Server is operational"
    CGroup: /system.slice/mysqld.service
            └─5884 /usr/sbin/mysqld

Dec 12 12:42:57 ip-172-31-73-244.ec2.internal systemd[1]: Starting MySQL Server...
Dec 12 12:43:05 ip-172-31-73-244.ec2.internal systemd[1]: Started MySQL Server.
[ec2-user@ip-172-31-73-244 ~]$
```


—> Conclusion

MySQL 8.0 was successfully installed and configured on the EC2 instance using the official MySQL community repository.

TASK 7 — Launch MySQL RDS image

1 Task Title

Launch a MySQL RDS Instance

2 Objective

To provision a fully managed MySQL database using Amazon RDS for use with applications or for migration from EC2-hosted MySQL.

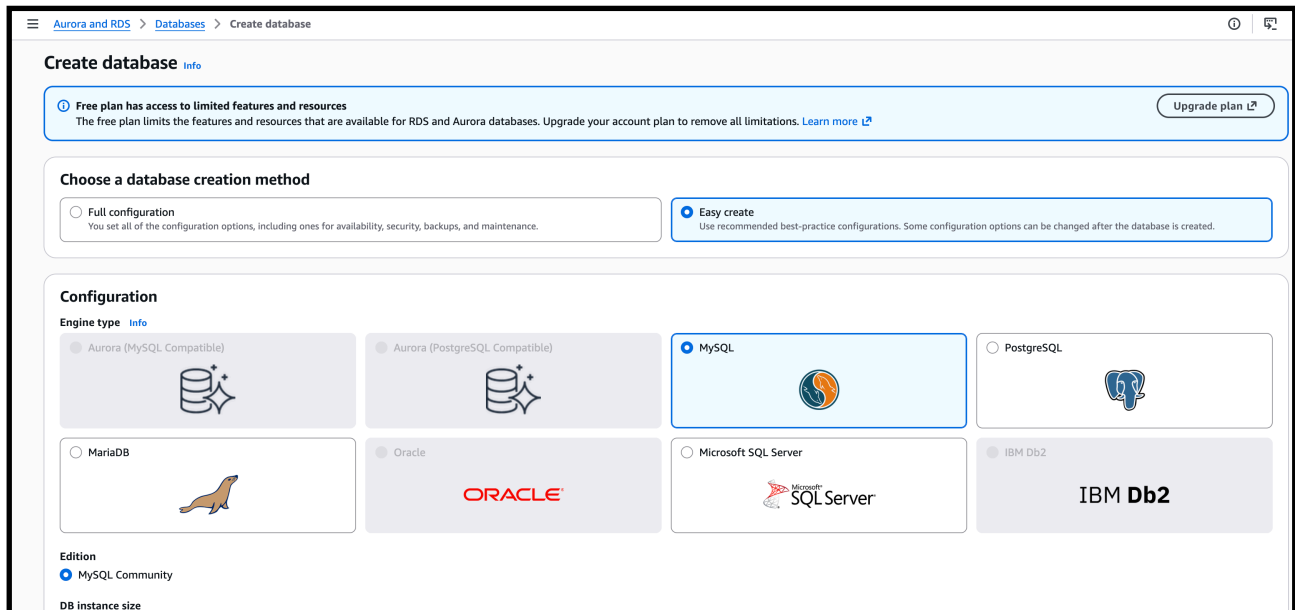
3 Prerequisites

- RDS Subnet Group created
- Security Group allowing port **3306**
- EC2 Bastion OR public access (optional)
- MySQL installed on EC2 (Task 6 completed)

4 Step-by-Step Implementation (With Evidence)

Step 1 — Open RDS Create Database

- **Console path:** AWS Console → **Services** → **RDS** → **Databases** → **Create database**.
- **Choose: Standard create.**



Step 2 — Engine selection

- **Engine type:** MySQL.
- **Version:** Choose a stable MySQL 8.x (select the latest minor you want, e.g., 8.0.x).
:camera_with_flash: **Evidence:** Screenshot showing MySQL engine/version selected.

Step 3 — Template & Settings

- **Template:** Free tier (if you want free-tier resources) or **Production** as required.
- **DB instance identifier:** mysql-instance-1 (or your naming).
- **Master username:** admin (or your chosen name).
- **Master password:** set and confirm a strong password (save it securely).

—> set a strong password in master password and conform that password

—> now it is finally available after 5 to 10 min off time

TASK 8 — Configure Multi-AZ

(Your exact task from the list)

① Task Title

Configure Multi-AZ for MySQL RDS Instance

② Objective

Enable Multi-AZ deployment for the existing RDS MySQL instance to ensure high availability and automatic failover across different Availability Zones.

③ Prerequisites

- RDS MySQL instance already created (Example: `database-2`)
- RDS status = **Available**
- Subnet group must have at least **2 AZs**
- IAM permission to modify RDS
- EC2 connectivity is already working
- Understanding: Multi-AZ is NOT free-tier (cost applies)

④ Step-by-Step Implementation (With Evidence)

Step 1 — Open the RDS Instance

Console path:
AWS Console → RDS → Databases → Select your DB

- ✓ Confirm:
- Status = **Available**
 - Engine = MySQL
 - Region = Same as EC2

Step 2 — Click Modify

Top right → **Modify**

Step 3 — Scroll to “Availability & durability”

You will see this option:

Multi-AZ deployment



AWS Free Tier does NOT allow Multi-AZ deployments.

Only **paid accounts** and **non-free-tier billing modes** get Multi-AZ.

—> **Configure Multi-AZ – Conclusion**

Multi-AZ deployment could not be enabled because my AWS account is on the **Free Tier plan**, and AWS does not allow Multi-AZ configuration for free-tier accounts.

The option “**Convert to Multi-AZ deployment**” is visible but **disabled**, showing the message:

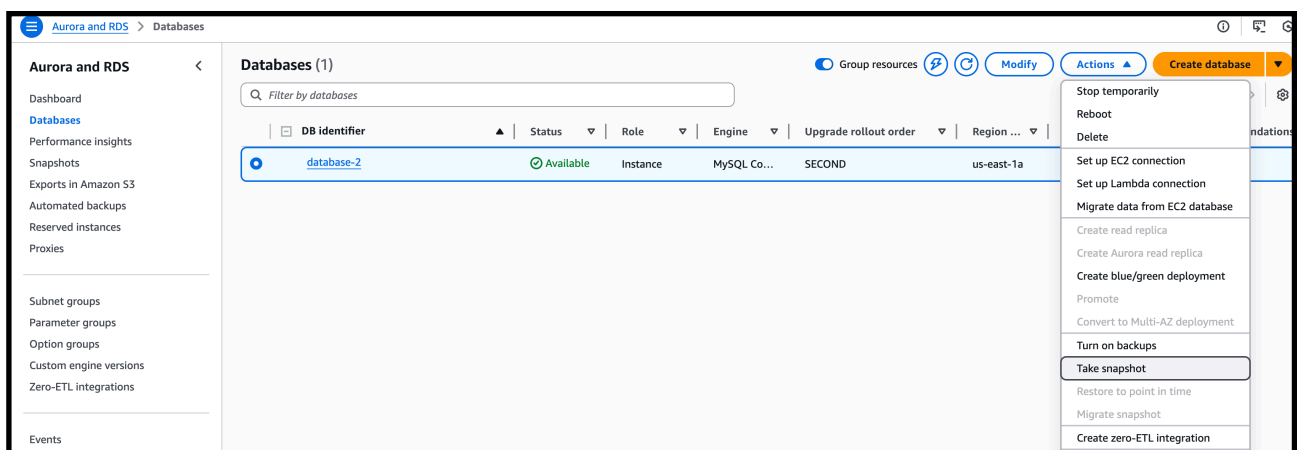
“This feature is not available with the free plan accounts.”

To complete Multi-AZ, a **paid AWS account** or **non-Free-Tier billing** is required.

TASK 9 — Take Backup of DB on EC2 & Restore it to RDS

Steps to follow

—> step 1 :~ RDS —> databases —> select the database —> click action —> take snapshot



—> give snap shot name

The screenshot shows the 'Take DB Snapshot' form in the AWS console. The form is titled 'Take DB Snapshot' and has a 'Preferences' section. Under 'Snapshot type', 'DB instance' is selected. Under 'DB instance', 'database-2' is selected. Under 'Snapshot name', 'snap-database2' is entered. The form also includes a 'Cancel' button and a 'Take snapshot' button.

Take DB Snapshot

Preferences
To take a DB Snapshot, choose a database and name your DB Snapshot.

Snapshot type
☒ DB instance
☐ DB cluster

DB instance
DB Instance identifier. This is the unique key that identifies a DB Instance.
database-2

Snapshot name
Identifier for the DB Snapshot.
snap-database2
Snapshot identifier is case insensitive, but stored as all lower-case, as in "mysnapshot". Cannot be null, empty, or blank. Must contain from 1 to 255 alphanumeric characters or hyphens. First character must be a letter. Cannot end with a hyphen or contain two consecutive hyphens.

[Cancel](#) [Take snapshot](#)

—> and click take snapshot

The screenshot shows the 'Snapshots' page in the AWS console. A green banner at the top indicates 'Successfully created snapshot snap-database2. View details'. The 'Manual' tab is selected, showing a table of manual snapshots. The table has columns for 'Snapshot name', 'Engine version', 'DB instance or cluster', 'Snapshot creation time', and 'DB Instance'. One snapshot is listed: 'snap-database2' with engine version '8.0.43', DB instance 'database-2', and creation time 'December 12, 2025, 19:08 (UTC+05:30)'.

Aurora and RDS > Snapshots

Successfully created snapshot snap-database2. View details

Snapshots

[Manual](#) [System](#) [Shared with me](#) [Public](#) [Backup service](#) [Exports in Amazon S3](#)

Manual snapshots (1)

Filter by manual snapshots

☒	Snapshot name	Engine version	DB instance or cluster	Snapshot creation time	DB Instance
☒	snap-database2	8.0.43	database-2	December 12, 2025, 19:08 (UTC+05:30)	December 12

—> we have successfully created a snapshot

Step 2 :~ lets delete the data base and restore it by snapshot

The screenshot shows the 'Databases' page in the AWS console. A green banner at the top indicates 'Successfully created snapshot snap-database2. View details'. The 'Databases' tab is selected, showing a table of databases. The table has columns for 'DB identifier', 'Status', 'Role', 'Engine', 'Upgrade rollout order', and 'Region'. One database is listed: 'database-2' with status 'Available', role 'Instance', engine 'MySQL Co...', and region 'us-east-1a'. A context menu is open over the 'database-2' row, showing options: 'Stop temporarily', 'Reboot', 'Delete', 'Set up EC2 connection', 'Set up Lambda connection', and 'Move data from EC2 database'.

Aurora and RDS > Databases

Successfully created snapshot snap-database2. View details

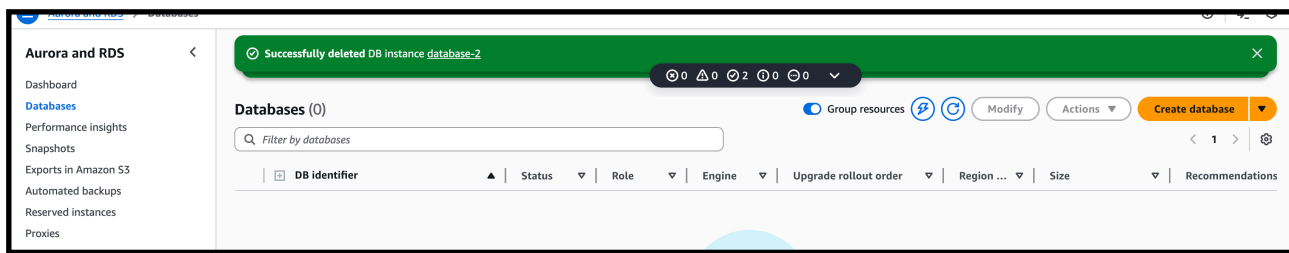
Databases (1)

Filter by databases

☒	DB identifier	Status	Role	Engine	Upgrade rollout order	Region
☒	database-2	Available	Instance	MySQL Co...	SECOND	us-east-1a

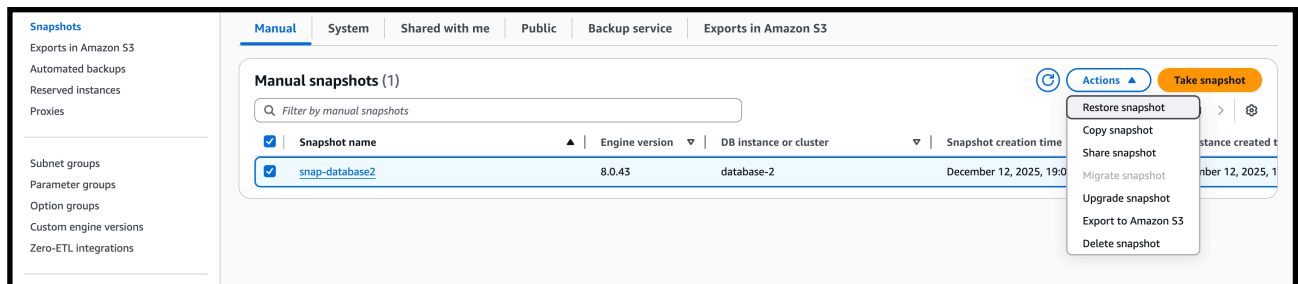
Stop temporarily
Reboot
Delete
Set up EC2 connection
Set up Lambda connection
Move data from EC2 database

—> after deleteing the data base

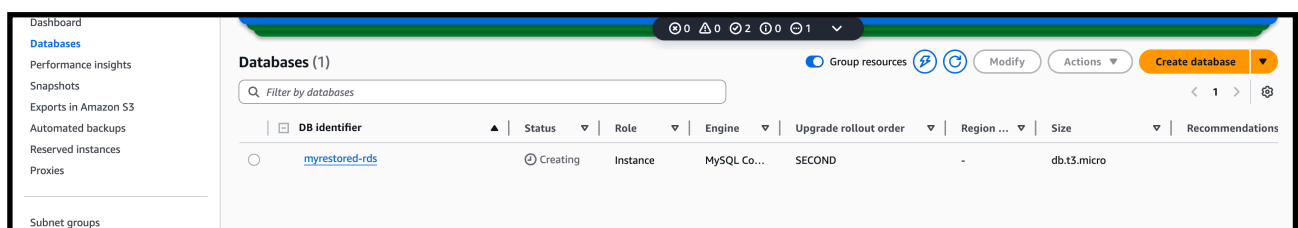


—> lets create the backup snapshot and relaunch the data base

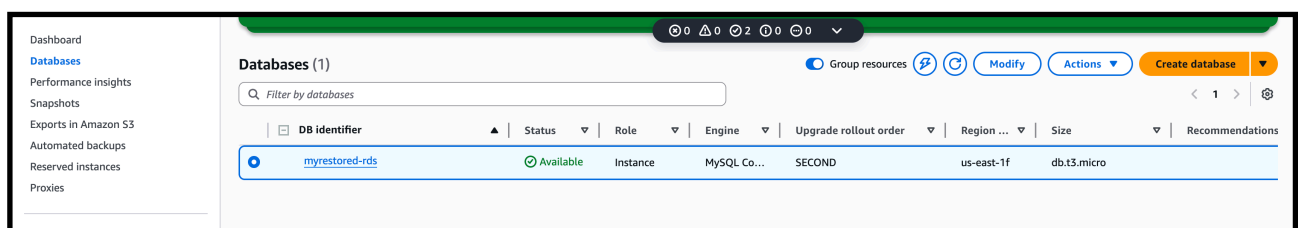
Step3:~ snapshot—> action—> restore snapshot



—> wait until the status is available when it is available we have successfully restored the data base



—> after waiting 10 min the status was available



Conclusion :~ successfully created a backup data of the data base

Task 10 — Create Read Replica

Objective

To create a Read Replica of the MySQL RDS instance for read-heavy workloads and to improve performance and availability.

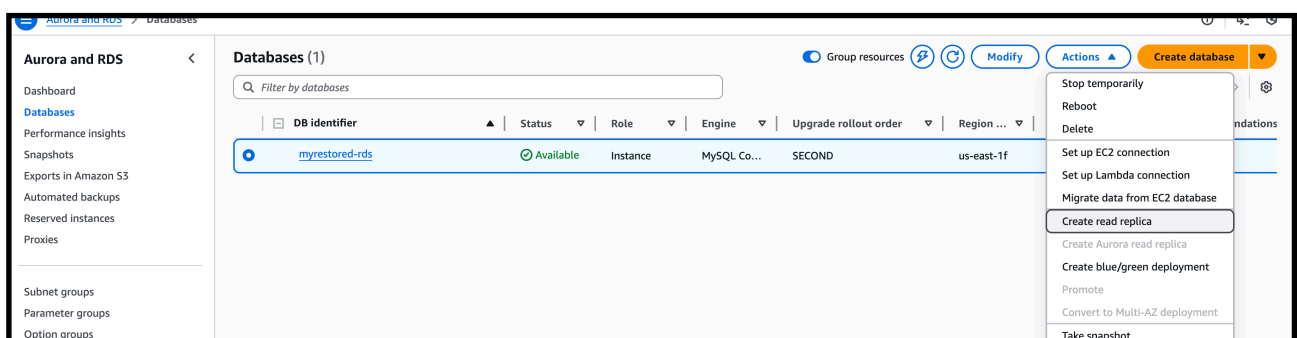
Steps to Create Read Replica

Step 1:

Database —> actions —> create read replica

Create Read Replica

1. Go to **RDS Console** → **Databases**
2. Select your DB (database-2)
3. Click **Actions** → **Create read replica**
4. In the configuration page:
 - Keep the same **engine version**
 - Choose the same **VPC**



Create read replica

You are creating a replica DB instance from a source DB instance. This new DB instance will have the source DB instance's DB security groups and DB parameter groups.

Settings

Replica source
Source DB instance identifier
myrestored-rds
Role: Instance

DB instance identifier
This is the unique key that identifies a DB instance. This parameter is stored as a lowercase string (for example, mydbinstance).
replica-db

Maintenance
Auto minor version upgrade
☒ **Enable auto minor version upgrade**
Enabling auto minor version upgrade will automatically upgrade your database minor version. For limitations and more details, see [Automatically upgrading the minor engine version documentation](#).
☐ **Enable deletion protection**
Protects the database from being deleted accidentally. While this option is enabled, you can't delete the database.

[Cancel](#) [Create read replica](#)

—-> and create the replica

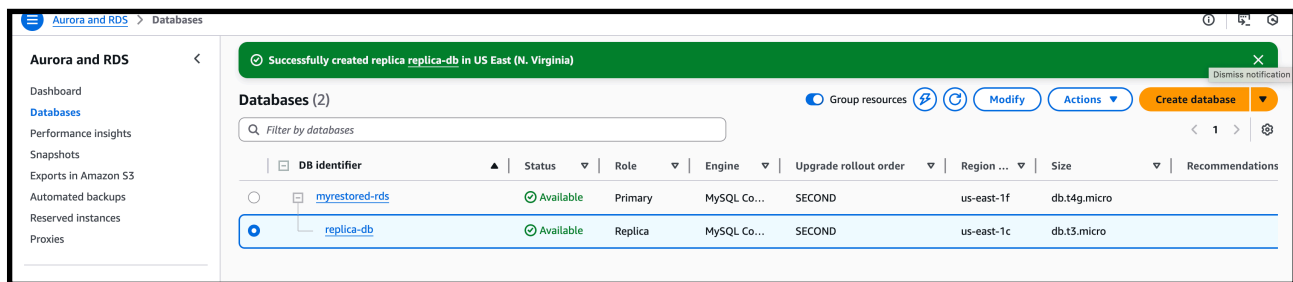
Step 2 — Verify the Read Replica

Once the replica is created:

1. Go to **RDS** → **Databases**
2. You will see a new DB with role:

Role: Read Replica

Aurora and RDS		Creating replica replica-db in US East (N. Virginia) Your database might take a few minutes to launch.							
Dashboard Databases Performance insights Snapshots Exports in Amazon S3 Automated backups Reserved instances Proxies		Databases (2) Group resources ⚡ ⌂ Modify Actions Create data							
Filter by databases									
DB identifier		Status	Role	Engine	Upgrade rollout order	Region ...	Size	Reco	
myrestored-rds		Modify...	Primary	MySQL Co...	SECOND	us-east-1f	db.t4g.micro		
replica-db		Creating	Replica	MySQL Co...	SECOND	-	db.t3.micro		



Conclusion

A read replica was successfully created , every modification takes time up to 15 min

